RATAN TATA MAHARASHTRA STATE SKILLS UNIVERSITY

School of Science Engineering and Technology Skills Department of Engineering

Subject –DevOps and Automation Lab

Lab Assignment 5

Aim:

Install Docker and run basic container operations.

Theory:

Introduction to Docker

1. Define Docker and explain its purpose in modern software development.

Docker is an open-source platform designed to automate the deployment, scaling, and management of applications using Containerization.

Purpose in Modern Software Development

- **Environment Consistency:** Docker ensures that an application runs the same way in development, testing, and production by packaging the code with its specific dependencies, libraries, and configuration files.
- **Isolation:** It allows multiple applications with conflicting requirements (e.g., different versions of Python or Java) to run on the same host without interference.
- **Microservices Architecture:** Docker is the foundational technology for microservices, allowing developers to break down monolithic applications into small, independent, and portable units.
- **Rapid Deployment:** Containers can be started or stopped in seconds, significantly increasing the speed of the CI/CD pipeline.

2. Difference between containers and virtual machines.

A Virtual Machine virtualizes the Hardware, whereas a Container virtualizes the Operating System.

Feature	Containers (Docker)	Virtual Machines (VMware/Hyper-V)
Operating System	Shares the **Host OS Kernel**.	Includes a full **Guest OS**.
Isolation	Process-level isolation.	Hardware-level isolation.
Size	Very Lightweight (MBs).	Heavyweight (GBs).
Startup Time	Milliseconds to Seconds.	Minutes.
Hypervisor	Not required (uses Docker Engine).	Requires a Hypervisor (Type 1 or 2).

3. Key components of Docker:

A. Docker Engine

The Docker Engine is a client-server application that acts as the core of the Docker system. It consists of:

- **Server (Daemon):** A long-running background process (dockerd) that manages Docker objects like images, containers, networks, and volumes.
- **REST API:** An interface that programs use to talk to the daemon and instruct it what to do.
- **Command Line Interface (CLI):** The client (docker command) that allows users to interact with the Docker Daemon.

B. Docker Images

A Docker Image is a read-only, executable package that includes everything needed to run an application.

- **Layered File System:** Images are built using a series of layers. Each instruction in a Dockerfile creates a new layer.
- **Immutability:** Once created, an image does not change. If you need to update the application, you build a new image.

C. Docker Containers

A Docker Container is a runtime instance of a Docker Image.

- **Writable Layer:** When a container is started, Docker adds a thin "writable layer" on top of the read-only image layers. All changes made to the running container (such as writing new files) are stored in this layer.
- **Ephemeral Nature:** Containers are generally designed to be ephemeral (temporary). Data intended to persist should be stored in Docker Volumes.

D. Docker Hub

Docker Hub is the official cloud-based Registry service provided by Docker for finding and sharing container images.

- **Public Repositories:** Contains official images for popular software like nginx, ubuntu, mysql, and python.
- **Private Repositories:** Allows organizations to securely store and share proprietary images within their teams.
- **Integration:** Docker Hub integrates with GitHub and Bitbucket to trigger automated builds when code is pushed to a repository.

Docker Installation

- Steps to install Docker on your operating system (Windows/Linux/Mac)
- System requirements for Docker installation
- Verify installation using command:
- *docker –version*

```
›› ~  12:50 docker --version
Docker version 29.2.0, build 0b9d198
```

Explain Docker Desktop.

Docker Desktop is an easy-to-install application for Mac, Windows, or Linux environments that enables you to build and share containerized applications and microservices. It provides a GUI (Graphical User Interface) to manage your containers, applications, and images directly from your machine.

Core Components

- **Docker Engine:** The underlying containerization technology including the daemon (dockerd) and CLI.
- **Kubernetes:** A certified, single-node cluster that can be enabled for local orchestration testing.
- **Docker Compose:** A tool for defining and running multi-container applications.
- **Docker Scout:** A built-in security tool that analyzes images for vulnerabilities.

- **Extensions:** A marketplace that allows you to integrate third-party developer tools (like databases or security scanners) into the interface.

How it Works

On Windows and Mac, Docker Desktop runs the Docker Engine inside a lightweight Virtual Machine (VM) (using Hyper-V/WSL2 on Windows or Apple Hypervisor on Mac). This allows Linux-native container technology to run seamlessly on non-Linux operating systems.

Basic Docker Commands

- *docker –help:*

  Displays a comprehensive list of all available Docker commands, management categories, and syntax options for the CLI. It can be appended to any specific command (e.g., docker run --help) to view detailed flags and usage instructions for that particular function.

```
❯❯ ~  12:50 docker --help
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  bake        Build from a file
  pull        Download an image from a registry
  push        Upload an image to a registry
```

- *docker info:*

  Provides a detailed summary of the system-wide configuration, including the number of running/paused containers, images, and the server version. It also displays critical host information such as the Storage Driver, Kernel Version, and available CPU/Memory resources.

```
❯❯ ~  12:52 docker info
Client:
 Version:    29.2.0
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.17.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.31.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.0.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
```

Docker command structure:

The Docker command structure follows the pattern: *docker [OPTIONS] COMMAND [ARG...],* where *COMMAND* can be a standalone command or a Management Command (like image or container) followed by a sub-command.

Working with Docker Images

- Pull an image from Docker Hub: *docker pull nginx*

```
❯❯ ~  12:55 docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
5e815e07e569: Pull complete
bb3d0aa29654: Pull complete
cde7a05ae428: Pull complete
510ddf6557d6: Pull complete
3189680c601f: Pull complete
ec781dee3f47: Pull complete
587e3d84dbb5: Pull complete
669e0ab8e7fa: Download complete
96a6cfe061e0: Download complete
Digest: sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

- List available images: *docker images*

```
❯❯ ~  13:00 docker images
                                                    i Info →   U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:latest         7150b3a39203        240MB         65.8MB
postgres:15-alpine   5fe8ca7fc662        392MB          110MB    U
```

- Remove an image: *docker rmi <image_id>*

```
❯❯ ~  14:44 docker rmi 7150b3a39203
Untagged: nginx:latest
Deleted: sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
```

Running Containers

- Run a container using: *docker run nginx*

```
❯❯ ~  13:01 docker run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/06 07:32:24 [notice] 1#1: using the "epoll" event method
2026/04/06 07:32:24 [notice] 1#1: nginx/1.29.7
2026/04/06 07:32:24 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/06 07:32:24 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/06 07:32:24 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/06 07:32:24 [notice] 1#1: start worker processes
```

- Run container in detached mode: *docker run -d nginx*

```
❯❯ ~  13:15 docker run -d nginx
662b94a8c29719fc08f88d85a97580781e23188117f30e32fa094fb96df0fd60
```

- Assign a name to a container
- Map ports (e.g., 8080 to 80

```
❯❯ ~  13:20 docker run --name test_nginx -p 8080:80 nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
```

```
❯❯ ~  03:22 docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS                      PORTS     NAMES
09fbeacac798   nginx                "/docker-entrypoint.…"   14 hours ago   Exited (0) 14 hours ago               test_nginx
```

Managing Containers

- List running containers: *docker ps*

```
❯❯ ~  13:03 docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED       STATUS                    PORTS                                          NAMES
c40ba146d067   postgres:15-alpine   "docker-entrypoint.s…"   13 days ago   Up 11 minutes (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp    dasf-postgres
```

- List all containers (including stopped): *docker ps -a*

```
❯❯ ~  13:03 docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS                     PORTS                                          NAMES
b34cb62db6fa   nginx                "/docker-entrypoint.…"   6 minutes ago   Exited (0) 5 minutes ago                                                  busy_lederberg
c40ba146d067   postgres:15-alpine   "docker-entrypoint.s…"   13 days ago     Up 16 minutes (healthy)    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp    dasf-postgres
```

- Stop a container: *docker stop <container_id>*

```
❯❯ ~  13:10 docker stop b34cb62db6fa
b34cb62db6fa
```

- Start a container: *docker start <container_id>*

```
❯❯ ~  13:08 docker start b34cb62db6fa
b34cb62db6fa
```

- Remove a container: *docker rm <container_id>*

```
❯❯ ~  03:46 docker rm b34cb62db6fa
b34cb62db6fa
```

Inspecting and Logs

- View container logs: *docker logs <container_id>*

```
›› ~  ☹ 13:15 docker logs b34cb62db6fa
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/06 07:32:24 [notice] 1#1: using the "epoll" event method
2026/04/06 07:32:24 [notice] 1#1: nginx/1.29.7
2026/04/06 07:32:24 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/06 07:32:24 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/06 07:32:24 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/06 07:32:24 [notice] 1#1: start worker processes
```

- Inspect container details: *docker inspect <container_id>*

```
›› ~  13:11 docker inspect b34cb62db6fa
[
    {
        "Id": "b34cb62db6fa63e16b19b91eeb7181e28b944d2d3ddef0097238206001558d92",
        "Created": "2026-04-06T07:32:23.884189194Z",
        "Path": "/docker-entrypoint.sh",
        "Args": [
            "nginx",
```

- Monitor container resource usage (optional): *docker stats*

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
b34cb62db6fa	busy_lederberg	0.00%	0B / 0B	0.00%	0B / 0B	0B / 0B	0

Executing Commands Inside Container

- Access running container: *docker exec -it <container_id> /bin/bash*

```
›› ~  13:14 docker exec -it b34cb62db6fa /bin/bash
root@b34cb62db6fa:/#
```

- Run simple commands inside container (e.g., ls, pwd)

```
›› ~  13:14 docker exec -it b34cb62db6fa /bin/bash
root@b34cb62db6fa:/# ls
bin  boot  dev  docker-entrypoint.d  docker-entrypoint.sh  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```